VERIQTA

★ SUBSCRIBER SERIES #2 ★

Real incident.
Real mistakes.
Real lessons.
Better systems.
Better outcomes.

THE DEPLOYMENT THAT BROKE PRODUCTION

A 10-PAGE CASE STUDY ON WHAT WENT WRONG,
HOW WE FOUND THE ROOT CAUSES,
AND HOW WE BUILT A BETTER WAY TO DEPLOY.

WHAT'S INSIDE

THE 5 MISTAKES
That caused the outage

INVESTIGATION TIMELINE
How we connected the clues

NEW CI/CD ARCHITECTURE
What we rebuilt and hardened

DEPLOYMENT CHECKLIST
What we check before every release

10 LESSONS LEARNED
Hard-earned lessons you can apply today

☆ OUTAGES ARE EXPENSIVE. LESSONS ARE FREE. LEARN, APPLY, LE

VERIQTA
Subscriber Series #2

VERIQTA

☆ Subscriber Series #2 ☆

New Series.
New Incident.
New Lessons.
Same Goal.
Better Systems. ☆

The CI/CD Pipeline That Deployed a Bug to Production

HOW A "SUCCESSFUL DEPLOYMENT" CAUSED A PRODUCTION OUTAGE

THE STORY

Friday. 4:37 PM.

CI pipeline passed.

Deployment succeeded.

All checks were green.

By 5:12 PM...

production was down. ☹

THE IMPACT ⚠

- 12,842 users affected
- Checkout failures
- Payments declined
- Estimated loss: $18,700
- Downtime: 35 minutes

THE HARD TRUTH

A "successful deployment" only means your code made it to production.

It does NOT mean your code is good. Or that your system can handle it.

OUR CI/CD PIPELINE (BEFORE THE OUTAGE)

1. **Code Commit** — Developer pushes code
2. **Build** — Code compiled, Build created
3. **Automated Tests** — All tests passed ✓
4. **Artifact Build** — Docker image created
5. **Deploy to Production** — Deployment succeeded ✓
6. **All Green!** — Pipeline successful ✓

OUTAGE STARTED — 5:12 PM

TIMELINE (WHAT HAPPENED)

4:37 PM	4:41 PM	4:43 PM	4:45 PM	4:47 PM	5:12 PM
Code committed to main	Build completed	Tests passed	Artifact pushed	Deployment succeeded	Users started reporting issues

WHAT WE THOUGHT

- ✓ Tests passed
- ✓ Build succeeded
- ✓ All systems green
- ✓ Safe to deploy

Looks successful. Feels good. But it was a trap.

WHAT ACTUALLY HAPPENED

- ✗ A bug slipped through
- ✗ No one caught it before release
- ✗ It affected real users
- ✗ The impact was immediate

We shipped the bug. Production paid the price.

THE LESSON WE LEARNED

A pipeline that only checks if you can deploy, is not enough. It must help you decide if you should.

The next 5 pages explain the 5 mistakes that led to this. →

The goal of CI/CD isn't speed.
The goal is safe, reliable, repeatable releases.

Every bug we shipped wasn't a surprise. It was a missing test.

VERIQTA

☆ Subscriber Series #2 ☆

Mistake #1

Missing Automated Tests

```
if (isValid() == false) {
  processOrder();
}
```

Logic bug in production

We had no safety net. The pipeline built the code, but never verified it.

WHAT HAPPENED

- Code was written in a hurry.
- No unit tests for critical logic.
- No integration tests.
- The bug made it all the way to production.

PIPELINE RESULT
✓ Code Commit
✓ Build
✓ Lint
✓ Package
✓ Deploy
ALL GREEN

The pipeline said "success".
Users said "this is broken".

THE IMPACT

Users unable to complete purchases	12,842 affected
Checkout failure rate spiked to	68%
Downtime	35 minutes
Estimated revenue loss	$18,700

A bug that could have been caught in seconds cost us thousands.

WHAT WAS MISSING

1. Unit Tests

We had zero tests for business logic.

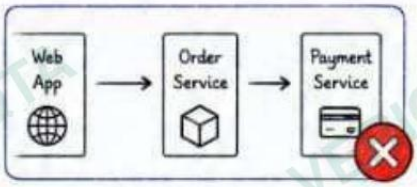
A small typo.
A wrong condition.
A huge impact.

```
// This should have failed the build
if (isValidPromo(code)) {
  applyDiscount();
} else {
  rejectPromo();
}
```

2. Integration Tests

We didn't test how services talk to each other.

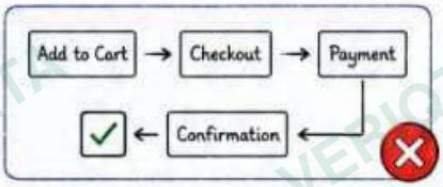
The bug happened in the real flow.



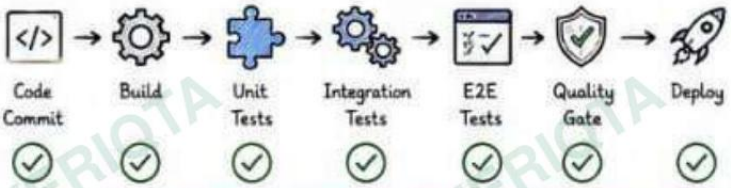
3. End-to-End Tests

We didn't test the real user journey.

If we did, we would have caught it.



HOW IT SHOULD HAVE BEEN



Bugs caught early are cheap.
Bugs caught in production are expensive.

KEY LESSON

A pipeline without tests is just a fast way to release problems.

- Automate tests.
- Make them part of the pipeline.
- Don't allow code to move forward without proof.

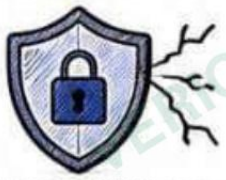
INTERVIEW NOTE

Q: How do you ensure bugs don't reach production?

A: I believe in testing at multiple levels: unit, integration, and E2E. I shift testing left and make quality everyone's responsibility.

Rushing code is normal. Rushing to prod is not.

Mistake #2
Weak Branch Protection



Anyone could merge
Anything could break

We allowed changes to go straight to main. No review. No guardrails.

WHAT HAPPENED

- Developers pushed code directly to main.
- No pull request.
- No mandatory code review.
- No checks before merge.
- The bug went from laptop to production in minutes.

Direct push = Direct problem

We trusted good intentions.
Production paid the price.

THE IMPACT

Users affected	12,842
Failed deployments	3
Time to detect	25 minutes
Estimated loss	$18,700

One unchecked change can impact thousands of users.

HOW IT WORKED (BEFORE VS. AFTER)

BEFORE (NO PROTECTION)

Developer Writes code → Push to main Direct push → CI/CD Pipeline Runs → Production Deployed

Too easy to break things.

AFTER (WITH PROTECTION)

Developer Writes code → Pull Request Open PR → Code Review Approve → CI/CD Pipeline Checks pass → Production Deployed

Slower by minutes. Safer for users.

WHAT GOOD LOOKS LIKE

- ☑ Pull requests required
- ☑ At least 1 approval required
- ☑ Status checks must pass
- ☑ No direct pushes to main
- ☑ Admin bypass is restricted
- ☑ Audit log of all changes

Branch protection on main

Require pull request	ON
Require reviews	ON 1
Require status checks	ON
Restrict pushes	ON
Allow force push	OFF

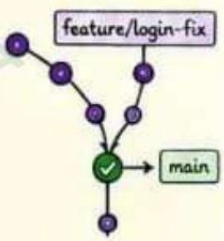
Good guardrails don't slow you down.
They keep you from breaking production.

KEY LESSON

Your process is part of your product.
Weak process = weak system.

Strong branch protection:

- ✓ Prevents accidental changes
- ✓ Catches issues before they ship
- ✓ Creates accountability
- ✓ Builds a culture of quality



INTERVIEW NOTE Q: Why is branch protection important?
A: It enforces quality gates. It ensures code is reviewed, tested, and safe before it reaches production.

Big changes need small steps. Not a big bang.

VERIQTA

☆ Subscriber Series #2 ☆

Mistake #3

No Canary Deployment

We released to everyone at once.

We skipped canary. We skipped safety. We skipped our users.

WHAT HAPPENED

- We deployed the new version to 100% of users.
- The bug was in a critical path.
- Everyone hit the bug at the same time.
- Our systems couldn't handle the spike in errors.
- Support was flooded.

100% USERS IMPACTED

A small bug.
A huge blast radius.

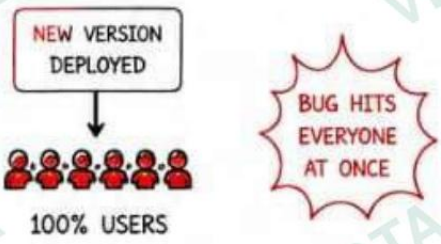
THE IMPACT

Users affected	12,842
Error rate	68%
Downtime	35 minutes
Estimated loss	$18,700

A bad release strategy turns small bugs into big outages.

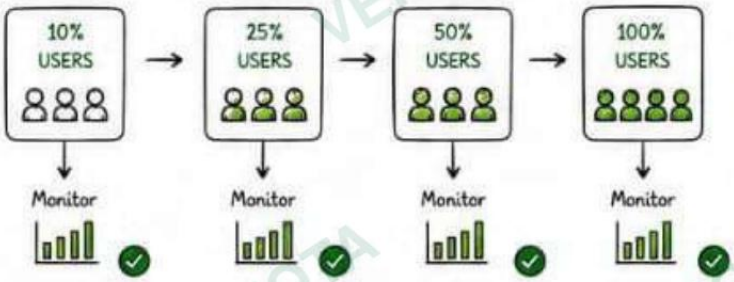
CANARY DEPLOYMENT: THE SMART WAY

BAD: BIG BANG DEPLOYMENT



High risk. High impact. No rollback window.

GOOD: CANARY DEPLOYMENT



Small steps. Early detection. Safe rollout.

HOW CANARY HELPED US (IN THE NEW PROCESS)

- ☑ Issues are detected early.
- ☑ Blast radius stays small.
- ☑ We can rollback before most users are impacted.
- ☑ We build confidence with data, not guesswork.

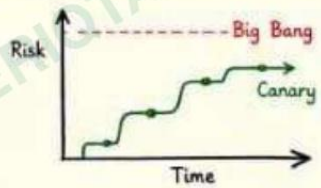
Canary turns deployment from a gamble into a controlled experiment.

KEY LESSON

Releasing to everyone is not bold. It's risky.

Use canary deployments.

- ✓ Start small
- ✓ Watch metrics
- ✓ Increase gradually
- ✓ Then go wide



INTERVIEW NOTE Q: What is canary deployment?

A: It's a strategy to release changes to a small subset of users first, monitor the results, then gradually roll out to everyone.

VERIQTA

☆ Subscriber Series #2 ☆

You don't plan to fail.
You fail to plan rollbacks.

Mistake #4

Rollback Was Broken

When rollback fails, outages multiply.

When things went wrong, our safety net had holes. Big ones.

WHAT HAPPENED

- We discovered the issue and decided to rollback.
- The rollback failed.
- The previous version wasn't available.
- Database migrations made it impossible to just "go back".
- Downtime extended from minutes to hours.

What should have been a 2-minute rollback turned into a 2-hour incident.

We had a plan on paper.
But it didn't work in reality.

THE IMPACT

Users affected	12,842
Failed rollback attempts	3
Additional downtime	95 minutes
Extra estimated loss	$28,300
Team stress level	High

Rollback is your last line of defense.
Ours was broken.

WHY THE ROLLBACK FAILED

1. Artifacts Overwritten
The previous build was overwritten by new deployments.

2. DB Migrations
Database schema changes made the old version incompatible.

3. No Blue/Green
Old Version → New Version
We deployed in-place. No separate environment to switch back.

4. Config Drift
Configuration changed during deployment. Rollback config mismatched.

5. Untested Rollback
We never tested rollback. We discovered problems in production.

A rollback strategy that isn't tested doesn't exist.

WHAT A GOOD ROLLBACK LOOKS LIKE

Version N (Previous) → Keep Artifacts Always → Database Backward Compatible → Blue/Green or Canary → Test Rollback Regularly

A rollback you can test is a rollback you can trust.

KEY LESSON

A deployment is only safe if you can return from it safely.

- Keep previous versions.
- Design backward compatible DB changes.
- Use blue/green or canary.
- Automate rollback.
- Test rollback in staging.

Plan for the worst so the worst doesn't become reality.

INTERVIEW NOTE

Q: How do you prepare for rollback?

A:
- I keep previous artifact versions.
- I make database migrations backward compatible.
- I use blue/green or canary deployments.
- I test rollback as part of release readiness.

WHERE WE ARE IN THE INCIDENT

1 Deployment Broke Prod → 2 Missing Tests → 3 Weak Branch Protection → 4 No Canary Deployment → 5 Rollback Was Broken → 6 No Release Monitoring → 7 Investigation Timeline → 8 New CI/CD Architecture → Checklist → Learned

VERIQTA

Subscriber Series #2

> You can't fix what you don't measure.
> You can't measure what you don't monitor.

Mistake #5

No Release Monitoring

We shipped blind.
Users paid the price.

Deployment succeeded. Alerts were silent. Users were screaming.

WHAT HAPPENED

- The deployment completed successfully.
- We assumed everything was fine.
- No one monitored the system after release.
- No dashboards. No alerts. No visibility.
- Users reported the issue on social media.
- We found out from our ussers, not our systems.

Green in the pipeline.
Red in reality.

We had data.
But no one was watching it.

THE IMPACT

Users affected	12,842
Time to detect (MTTD)	35 minutes
Time to resolve (MTTR)	95 minutes
Estimated loss	$18,700
Customer trust	Damaged

The problem wasn't just the bug.
It was our blindness.

WHAT WE SHOULD HAVE BEEN MONITORING

1. ERROR RATE

Spike in errors should have alerted us within seconds.

Example Alert:

Error rate > 1% for 2 minutes

2. RESPONSE TIME

Users felt the slowness before the errors became obvious.

Example Alert:

P95 latency > 500ms for 2 minutes

3. FAILED REQUESTS

Failed requests are often the first signal of trouble.

Example Alert:

5xx errors > 10 for 1 minute

4. INFRA HEALTH

CPU, Memory, DB connections should never surprise us.

Example Alert:

CPU > 80% or DB connections > 80%

5. BUSINESS METRICS

Revenue, signups, conversions tell the real impact story.

Example Alert:

Conversions drop > 20% in 5 minutes

WHAT GOOD RELEASE MONITORING LOOKS LIKE

Deploy → Monitor in real-time → Alert immediately → Respond fast → Resolve quickly

Good monitoring doesn't prevent bugs.
It prevents small bugs from becoming big outages.

KEY LESSON

If you don't monitor your releases, you're gambling with your users.

- Monitor every release.
- Create dashboards before incidents happen.
- Set alerts that matter.
- Watch business metrics, not just technical ones.
- Know fast. Respond fast. Resolve faster.

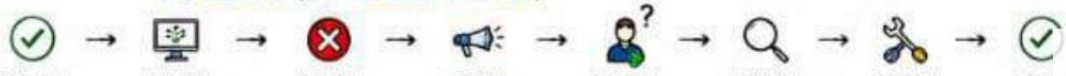
INTERVIEW NOTE

Q: What is the goal of release monitoring?

A: Detect issues early, minimize impact, and maintain user trust.

Visibility gives you options.
Blindness takes them all.

THE REALITY (WHAT ACTUALLY HAPPENED)



5:12 PM	5:20 PM	5:32 PM	5:40 PM	5:47 PM	5:58 PM	6:47 PM	6:57 I
Deployment Succeeded	First User Error	Social Media Complaints	Support Team Alerted	Engineering Notified	Issue Detected	Fix Deployed	Syste Recovered

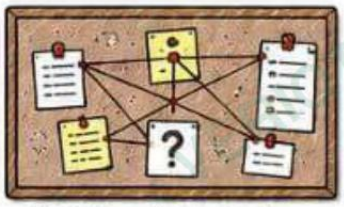
35 MINUTES

The truth is in the timeline. Every clue mattered.

☆ Subscriber Series #2 ☆

Investigation Timeline

Connecting the Clues



We followed the trail.
The system told the story.

Incidents don't happen in a vacuum. They leave clues.

THE INVESTIGATION TIMELINE

1. DEPLOYMENT SUCCEEDED

5:12 PM

CI/CD pipeline completed successfully.

All checks green.

2. FIRST USER COMPLAINTS

5:20 PM

Users reported issues on social media.

"Checkout not working!"

3. SUPPORT TICKETS SPIKE

5:28 PM

Support tickets jumped from normal ~3/min to 42/min.

Something is very wrong.

4. ENGINEERING ALERTED

5:35 PM

On-call engineer acknowledged the alerts.

War room activated.

5. INVESTIGATION BEGINS

5:40 PM

Logs, metrics and traces collected from all systems.

Searching for answers.

6. ROOT CAUSES IDENTIFIED

6:07 PM

We connected the dots and found all 5 root causes.

The real story emerged.

WHAT WE DISCOVERED

1. Bug introduced in checkout service.
2. Code merged directly to main without review.
3. No automated tests caught the bug.
4. Deployed to 100% of users (no canary).
5. Rollback failed due to incompatible DB schema.
6. No monitoring meant we didn't detect early.

5 ROOT CAUSES
1 ✓
2 ✓
3 ✓
4 ✓
5 ✓

One small bug.
Five process failures.
One big outage.

EVIDENCE WE COLLECTED

- Application logs
- Metrics & dashboards
- Distributed traces
- Database logs
- Support conversations
- User tickets & complaints

Data doesn't lie.
It points you to the truth.

THE KEY BREAKTHROUGH

The investigations showed a pattern, not just a problem.

Weak Process → No Visibility → Risky Deployment → Slow Recovery → Big Impact

We didn't just fix the bug.
We fixed the system that allowed it to happen.

LESSON FROM THE TIMELINE

Every minute matters in an incident.

- ✓ Early detection reduces impact.
- ✓ Good data shortens investigation.
- ✓ Clear timeline prevents repeat incidents.

Speed comes from preparation.
Preparation comes from process.

INTERVIEW NOTE

Q: What was the hardest part of the investigation?

A: Accepting that the problem wasn't a single bug. It was a chain of process failures.

Good engineers find bugs. Great engineers find why the system allowed the bug.

VERIQTA

Subscriber Series #2

The New CI/CD Architecture

What We Rebuilt. What We Hardened.

Good architecture is built on lessons learned, not assumptions.

A strong pipeline prevents weak releases.

We didn't just fix the bug. We rebuilt the system.

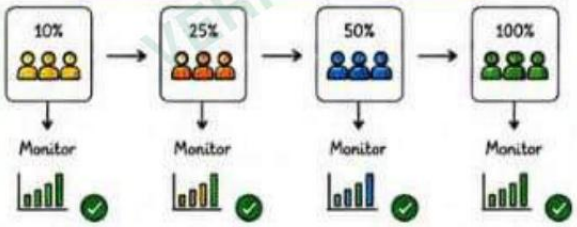
THE NEW PIPELINE (END-TO-END)

1	Code Commit	Developer pushes code to GitHub.	git push origin feature/login-fix ✓ Commit 7f3a2c1
2	Pull Request	PR created with details and test plan.	✓ PR #324 opened ✓ Checks triggered
3	Code Review	Peer review required. Knowledge shared.	✓ 2 approvals required ✓ Conversations resolved
4	Automated Tests	Unit, integration and contract tests run.	✓ 128 tests passed ✓ 0 failed
5	Security Scan	SAST, dependency scan and secret scan.	✓ No critical issues ✓ 3 low issues (ignored)
6	Build & Package	Build artifact created and versioned.	artifact: app-1.4.7 ✓ Stored in VERIQTA
7	Deploy to Staging	Deployed to staging environment.	✓ Smoke tests passed ✓ UAT ready
8	Canary Release	Release to 10% of users. Monitor closely.	10% → 25% → 50% → 100% Metrics look good
9	Release Monitoring	Track metrics, logs and user behavior.	✓ Error rate normal ✓ Latency normal
10	Production Rollout	Full rollout with confidence and visibility.	✓ 100% users ✓ Release successful

THE GUARDRAILS WE ADDED

Branch Protection	No direct pushes to main. PR + review required.
Required Approvals	Minimum 2 approvals for all changes.
Quality Gates	Tests, security and quality must pass.
Rollback Strategy	One-click rollback. Tested every sprint.
Dashboards	Real-time visibility into health and performance.
Alerts	Smart alerts for issues that matter.
Runbooks	Documented steps for common scenarios.

SAFE DEPLOYMENT STRATEGY



Proceed only when metrics are healthy.

WHAT CHANGED (BEFORE vs. AFTER)

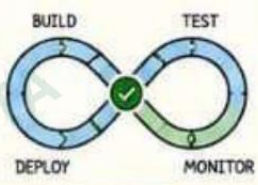
BEFORE (WHAT FAILED US)	AFTER (WHAT PROTECTS US)
✗ No tests	✓ Automated tests everywhere
✗ Direct pushes to main	✓ PR + review + approvals
✗ 100% rollout at once	✓ Canary rollouts
✗ Rollback not tested	✓ Rollback tested regularly
✗ No monitoring	✓ Real-time monitoring
⑩ We flew blind	✓ We ship with confidence

KEY LESSON

A strong pipeline is a force multiplier.

It helps good teams ship fast, safely, and reliably.

- ✓ Build quality in, not bolt it on.
- ✓ Automate the repeatable.
- ✓ Add guardrails, not roadblocks.
- ✓ Observe everything. React fast.
- ✓ Trust, but verify. Always.



INTERVIEW NOTE Q: What makes a CI/CD pipeline effective?

A: It's not the tools. It's the process, the guardrails, the visibility, and the culture behind it.

Great pipelines are designed. Not accidental.

WHERE WE ARE IN THE INCIDENT

Deployment Broke Prod → Missing Tests → Weak Branch Protection → No Canary Deployment → Rollback Was Broken → No Release Monitoring → 7 Investigation Timeline → 8 New CI/CD Architecture → 9 Readin Checklist

VERIQTA

Subscriber Series #2

Deployment Readiness Checklist

Don't learn these lessons in production. Use this list. Every time.

Great deployments are prepared, not lucky.

A checklist is your safety net. Use it every release.

1. CODE QUALITY

- [ ] **Tests Passing** — All unit, integration and contract tests pass.
- [ ] **Code Reviewed** — Peer review completed. Feedback addressed.
- [ ] **Static Analysis** — Code quality checks passed (lint, complexity, security hotspots).
- [ ] **Security Scans** — No critical or high vulnerabilities.

2. DEPLOYMENT SAFETY

- [ ] **Canary Configured** — Canary plan defined (%, metrics, duration).
- [ ] **Rollback Tested** — Rollback procedure tested in non-prod environment.
- [ ] **Staging Validated** — Deployed to staging. Smoke tests passed. Business flows verified.
- [ ] **Data Changes Ready** — DB migrations backward compatible and tested.

3. MONITORING & ALERTS

- [ ] **Dashboards Ready** — Release dashboards created and verified.
- [ ] **Alerts Enabled** — Critical alerts enabled and routed to on-call.
- [ ] **SLIs/SLOs Defined** — Error rate, latency and availability thresholds confirmed.
- [ ] **Synthetic Checks** — Synthetic monitoring active for key flows.

4. OPERATIONS READINESS

- [ ] **Runbook Updated** — Runbook reflects changes and rollback steps.
- [ ] **Team Informed** — Stakeholders and on-call team notified.
- [ ] **Incident Contacts** — Escalation path and contact list accessible.
- [ ] **Change Recorded** — Change ticket created with risk and impact.

5. RELEASE VALIDATION

- [ ] **Acceptance Criteria** — All acceptance criteria met and validated.
- [ ] **Feature Flags** — Feature flags configured and tested (if used).
- [ ] **Dependencies Check** — Third-party services healthy and compatible.
- [ ] **Release Window** — Within agreed release window and freeze rules.

6. COMMUNICATION & DOCUMENTATION

- [ ] **Release Notes Ready** — What changed, why it changed, and impact.
- [ ] **Status Page Ready** — Status page message prepared (if needed).
- [ ] **Knowledge Shared** — Key changes shared with support and ops.
- [ ] **Artifacts Stored** — Build artifacts and release packages are stored.

FINAL READINESS GATE – BEFORE YOU DEPLOY

- Can we detect failure in 1 minute? — Monitoring and alerts are active and tested.
- Can we rollback in 5 minutes? — Rollback tested and proven to work.
- Can we explain this release? — We know what changed, why, and the risk.
- Would we deploy this on Friday at 4:30 PM? — If not, it's not ready.

If you can't check every box above, pause and fix it.
Speed without safety is a gamble.

INTERVIEW NOTE

Q: Why is a deployment checklist important?

A: It reduces human error, standardizes the process, and ensures nothing critical is missed under pressure.

PRO TIP

Make this checklist a habit. Great teams use it every time, not just during big releases.

WHERE WE ARE IN THE INCIDENT

1 Deployment Broke Prod → 2 Missing Tests → 3 Weak Branch Protection → 4 No Canary Deployment → Rollback Was Broken → 6 No Release Monitoring → Investigation Timeline → 7 Investigation Timeline → 8 New CI/CD Architecture → Checklist → Learned

VERIQTA

☆ Subscriber Series #2 ☆

We didn't get lucky.
We got better.

10 Lessons Every DevOps Engineer Learns About Deployments

Better deployments build trust.
Trust builds products.

Outages are expensive. Lessons are free. Learn, apply, and level up.

1 Pipelines Don't Guarantee Good Releases.

A green pipeline only means the code was deployed.

Success in CI/CD is not success in production.

2 Automated Tests Are Safety Nets.

Tests catch defects early when they are cheap to fix.

Test everything that can break in production.

3 Process Protects Production.

Reviews, approvals and guardrails stop risky changes.

Strong process beats individual heroics.

4 Canary Releases Reduce Blast Radius.

Test with a small group first. Validate, then expand.

Small steps prevent big outages.

5 Rollback Must Be Tested.

A rollback that fails is worse than no rollback at all.

Test rollback like you test deploy.

6 Monitoring Starts at Deployment.

If you don't watch the release, you're flying blind.

Visibility turns incidents into quick fixes.

7 Incidents Leave Clues.

Timelines, data and patterns reveal the real story.

Follow the clues, not the blame.

8 Architecture Is Prevention.

Good architecture builds resilience into your delivery system.

Design for failure. Plan for recovery.

9 Checklists Prevent Repeat Failures.

A checklist makes sure nothing important is forgotten.

Don't rely on memory. Rely on process.

10 Great Deployments Are Designed, Not Hoped For.

Reliability is built with intent, practice and discipline.

Ship with confidence. Sleep with peace.

THE JOURNEY WE TOOK

Every failure taught us something. Every lesson makes us better.

WHAT WE BUILT TOGETHER

- A culture of quality and ownership
- A pipeline with guardrails and visibility
- Faster releases with less risk
- Systems that detect, recover and improve
- A team that learns and grows together

Better pipeline. Better process.
Better products. Better engineers.

> We didn't just fix a bug.
> We built a better delivery system.

Keep learning.
Keep improving.
Keep shipping safely.

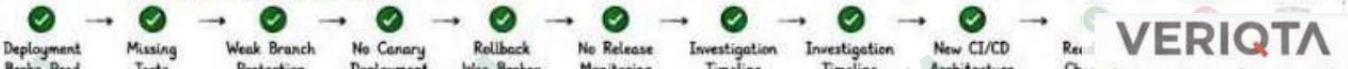
FINAL INTERVIEW NOTE

Q: What's the most important lesson you learned from this incident?

A: Systems fail. Teams learn. Engineers build better.

It's not about avoiding failures. It's about building systems that recover and keep improving.

WHERE WE ARE IN THE INCIDENT



Share this with your team. Save it for your next release. Make it a habit.

Be prepared today. Stay reliable tomorrow.